

Operit2 技术白皮书

个人设备空间中的持续 Agent 核心

第一版

作者 Operit 团队

2026 年 9 月



摘要

Operit2 面向一个日常问题：人的设备越来越多，能力分散在不同终端，而 Agent 的上下文、行动和结果往往局限于一台机器或一次会话。我们希望把手机、桌面、浏览器和云端设备连接成属于用户的个人设备空间。每台设备保留自己的系统能力和权限边界，Agent 则在设备之间延续任务、上下文与用户意图。

这份白皮书记录 Operit2 的工程判断和长期方向。我们关心的是：当模型、提示词、工具规划和交互方式不断改变时，用户的对话、任务、数据、设备关系和授权决定能否延续。为此，我们希望让模型适配与运行时保持可替换，同时把节点身份、Host 能力、持久化记录、同步规则 and 用户控制作为需要长期维护的基础。

Operit2 目前处于预览阶段。本文以已有源码和架构文档为工程基线，并明确区分当前基础与演进方向：节点连接、持久化同步和运行时已有实现；能力调度、插件跨节点接续与完整恢复体验仍需完善。研究引用用于解释设计思路，不构成性能验证或行业比较。

内容目录

- 一 项目起点与问题重构
- 二 面向个人的 Agent 设备空间
- 三 Operit2 的工程架构
- 四 对话主权 任务执行与上下文连续性
- 五 设备能力 健康与调度
- 六 权限 同步与用户主权
- 七 插件化与可演进的运行时
- 八 研究语境与技术立场
- 九 可靠性 迁移与预览期演进
- 十 实施重点与结语
- 附录 源码基线与参考资料



项目起点与问题重构

Operit 的第一代产品从 Android Agent 出发。团队没有把手机当成一个只负责显示对话的薄客户端，而是把本地 Linux 执行环境、终端会话、文件系统和移动端权限能力一起带进 Agent 的工作范围。历史代码中可以看到 PRoot 驱动的 Ubuntu 环境、可见 PTY 会话、后台复用 shell，以及通过 Android Storage Access Framework 暴露 Ubuntu 文件系统的实现。这段经历让我们很早面对一个现实：设备的形态不应决定它在 Agent 世界里的价值。[2]

在积累了较深入的移动端 Agent 实践后，Operit2 最初着眼于跨平台扩展。但让同一套对话和工具出现在更多终端，还不足以解决使用中的割裂：手机经常随身携带，却不适合承载所有重任务；桌面适合创作和审核，却未必随时可用；云端设备可以长期在线，但不应因此获得身份上的特权。我们逐渐把重点转向一个问题：如何让 Agent 的任务与上下文在一个人的设备之间延续。

这种问题重构也决定了 Operit2 的产品范围。我们优先服务个人，而不是把设备协作改造成组织治理系统。一个人可以拥有多台设备，也可以把云服务器、浏览器或新购买的电脑加入自己的空间；但设备协作不等于多人审批链、企业组织结构或管理层级。Operit2 希望降低普通人接近 AI 能力的操作门槛，让复杂的设备能力可以被理解、被授权、被调度，而不是要求每位用户先学会终端、环境变量和部署拓扑。

面向个人的 Agent 设备空间

我们把运行 Operit Core 的一个实例称为 CoreNode。它可由支持的移动端、桌面或云端运行环境承载；浏览器运行时的能力和接入方式需要单独区分。每个节点通过自己的 HostManager 使用本机 Host 提供的文件、终端、浏览器、网络、模型、存储和平台交互能力。节点的协作角色应由可用性、能力和用户策略决定，而非由操作系统或设备类别预先指定。



图 1 个人 Space 中平等 CoreNode 的协作关系

Space 是多个已经建立信任关系的 CoreNode 所组成的协作范围。它首先解决的是哪些节点共享持续同步的业务记录，哪些节点能够承载路由流量。它不是一台中心服务器，也不是把远端的文件系统、管理员权限或正在运行的进程直接暴露给其他节点。节点的本地能力始终由它自己的 Host 管理；跨节点协作是让合适的节点在合适的上下文中执行，而不是把所有能力复制到发起端。

概念	在个人空间中的职责	必须保持的边界
CoreNode	运行 Core 与本地 Host 的独立节点，可以承担对话、工具或持续任务。	节点之间地位对等，长期在线不等于中心化。
Host	提供本地文件、终端、浏览器、网络、模型与系统接口。	系统权限由本机和部署环境决定，不会随同步被继承。
Space	组织成员、持久化同步范围与可路由的协作关系。	不替代本地执行环境，也不等同于共享 root 权限。
Link 和 PeerLink	处理配对、认证、实时调用、观察流和多跳转发。	两两信任是直接网络连接的基础。
Binding	记录某项可执行工作下一段应该由哪个节点继续。	改变执行位置，不把聊天或 Agent 数据搬迁成单点所有。

Operit2 的工程架构

Operit2 以 Rust Core 为基础，通过平台 Host、Flutter 应用、CLI 和 Web Access 接入。Core workspace 按 Host API、基础模型、持久化存储、工具、插件、模型 Provider、运行时、节点路由、Access、Link 与



代理生成划分职责，部分 crate 的拆分与生命周期管理仍在调整。上层运行时负责组装实现，底层通过数据结构和 trait 暴露能力，减少平台适配、模型接入和前端之间的耦合。[1]

不同模块需要不同的演进节奏。模型 Provider、提示词、工具规划、UI 和脚本运行时会快速变化；Host API、存储边界、节点身份、同步记录和权限语义则需要谨慎维护。Core 将 Host API、数据模型与存储放在下层，再组合工具、Provider、JavaScript bridge 和应用运行时，为替换模型、增加平台或调整交互方式保留空间。

持久化同步与实时传输

在节点协作中，Operit2 有意把两条数据路径分开。SpacePersistenceSyncService 订阅本地持久化变更，在直接配对的节点之间交换同步操作、向量时钟和必要的内容寻址文件。聊天、偏好、对象数据、Binding 与运行时文件可以按声明的持久化范围同步。节点身份私钥、两两配对密钥、HostManager 平台状态、本机监听地址和临时交互句柄留在本地。这样的分离让恢复和继续成为可设计的问题，也避免把短暂连接状态误认为可复制的业务事实。

实时路径承担 Core call、watch、push、流事件、Agent continuation 与 Host 交互响应。CoreNodeRouter 根据路由元数据决定请求是在本机执行，还是经由 PeerLink 到达目标 CoreNode。实时传输可以等待持久化同步达到某个时钟位置，但这个屏障只保证目标节点看到足够的任务事实，并不意味着正在运行的模型请求、PTY、浏览器会话或进程句柄被原样迁移。

Host 是能力边界

HostManager 将文件系统、终端、HTTP、浏览器自动化、浏览器会话、存储、密钥保管、运行时调度和本地推理等能力以 trait 方式交给 Core。HostEnvironmentDescriptor 同时描述平台、当前权限、隔离状态、结构化能力和工作区根目录。Windows、Linux、Android、OpenHarmony、Apple 与 Web Host 可以通过这一边界暴露不同能力，而上层 Agent 不需要把平台差异硬编码进对话逻辑。

浏览器在 Operit2 中也不只是一个访问页面。Web Access 可以作为一个已运行节点的浏览器访问面；浏览器平台 Host 则有独立的运行时和能力描述。两者不能混为一谈。前者让用户在浏览器中接近自己的 Agent，后者为浏览器成为独立运行节点留下工程路径。

对话主权 任务执行与上下文连续性

如果多台设备同时把即时输入注入同一个模型上下文，任务顺序与执行归属就容易混乱。Operit2 的设计将主对话与执行过程分开：主对话保留发起设备的交互归属，执行节点负责任务或子任务的局部工具循环，Space 异步同步状态、结果、产物与可恢复记录。其他设备可以查看进展，但同步并不要求把所有过程实时注入同一轮模型请求；离线节点也需要在恢复连接后追上状态。

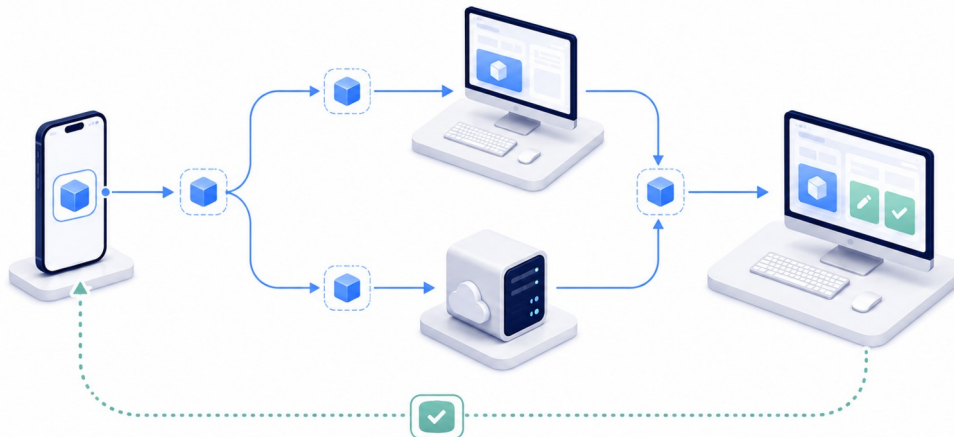


图 2 任务在发起 执行 同步与回看之间保持连续

Binding 记录下一段工作由哪个 CoreNode 继续，并使用递增的 generation 标识执行归属的变化。交接选择在工具结果已持久化、下一轮模型请求尚未开始的边界进行；目标节点取得必要记录后接续，旧节点应停止推进。这个机制用于约束同一轮的执行者，但不能单独保证外部动作只发生一次。网络故障后的重试还需要结合幂等性、去重和结果核验，避免重复提交。

从临时运行留下可恢复的任务事实

shell、流连接、浏览器会话和模型请求包含大量瞬时细节。当前接续机制不迁移这些运行实例，而是保存足以理解和继续工作的任务事实：执行节点、时间、工具参数、主要动作、结果、产物和必要的错误信息。阶段总结便于下一台设备快速了解进度，原始日志与产物则可按保留策略作为详细证据。总结不可避免地省略细节，因此恢复能力取决于实际保存的记录，不能仅凭一段摘要承诺无损恢复。

信息层	典型内容	在协作与恢复中的作用
实时运行层	进程句柄、PTY、流连接、短期缓存、UI 状态。	由当前节点管理，连接中断时允许结束或重建。
任务轨迹层	工具参数、执行节点、重要事件、结果、错误、产物引用。	可同步、可审计、可在另一节点上继续理解。
语义历史层	阶段总结、用户意图、决定理由、可继续的下一步。	控制主模型上下文密度，支撑备份和恢复。
本地敏感层	设备私钥、配对密钥、Host 临时权限与系统句柄。	保留在节点本地，不作为默认同步内容。



设备能力 健康与调度

Linux 云端设备在个人 Space 中是普通 CoreNode。它可能因长期在线、算力或服务条件合适而承担更多工作，这是任务分配的结果，并非系统赋予的中心身份。新电脑、新服务器以及未来接入 Space 的独立浏览器节点，都应遵循相同原则：保留各自能力，根据任务需要和用户选择参与协作。

能力调度仍是需要设计与验证的方向。我们计划结合三类信息选择节点：加入时声明的平台、硬件、运行时和服务；当前可达性、资源压力与连接质量；长期使用中积累的执行耗时和成功、失败记录。一次失败只是一条证据，不应永久否定某项能力；新增 GPU、安装服务或网络变化，也应及时更新能力判断。下表描述调度需要考虑的维度，不代表完整机制已经实现。

能力视角	回答的问题	调度含义
声明能力	这台设备声称拥有什么平台、运行时、硬件和服务。	用于初始匹配和兼容性筛选。
实时健康	它现在是否可达、可接收任务、资源是否紧张。	用于短期可用性判断，具有时效性。
长期证据	它是否成功完成过这类工作，耗时与稳定性如何。	积累实际能力的证据，并随环境变化修正判断。
用户策略	隐私、费用、耗电、优先设备、云端使用和冗余偏好。	将效率置于用户选择之后。

我们考虑把冗余用于备用切换、受控并行和子任务拆分，具体策略尚待验证。并行工作需要符合权限、费用和资源限制；涉及外部写入的任务尤其需要明确执行归属，并处理幂等性、去重和失败核验。租约或 generation 只能构成其中一部分，不能替代对外部副作用的控制。目标是让设备在合适的时候参与，而非让所有节点长期满载。

权限 同步与用户主权

个人 Agent 的核心不是替人做所有决定，而是让人能看见、理解并决定重要边界。Operit2 的权限架构将外部真实隔离、Host 被系统授予的能力、AI 当前能力模式和具体工具调用批准从上到下区分开来。下层只能继续收紧能力，不能绕过上层。完整权限不等于 root 或管理员提权；设备进程没有的系统能力，Agent 也不能凭一个应用内开关获得。



层级	责任	用户能理解的含义
运行环境隔离	虚拟机、容器、系统账号、Android 应用沙盒等外部边界。	设备或部署环境本身允许什么。
Host 授权	操作系统或部署环境授予当前 Host 的真实能力。	这台机器和这个账号真正能做什么。
AI 能力模式	只读、工作区读写或完整权限等运行时限制。	此刻 AI 被允许读、写和调用到什么范围。
具体工具批准	对一次有影响的工具调用进行确认。	哪一个动作会在何处发生，是否由我授权。

敏感同步的交互仍需完善。我们希望用户能够看清要同步的对象、目标节点和后果，再决定同步或留在本机；撤销授权、轮换凭据与恢复数据也应有易用的入口。撤销后续访问不等于收回已经复制的数据，系统需要说明这种区别。命令行和环境变量不应成为理解这些选择的前提，最终决定权始终属于用户。

插件化与可演进的运行时

模型适配、提示词、工具规划、UI 和执行器都可能随着模型能力与使用方式而改变。Operit2 已提供 ToolPkg、Skill、MCP、Compose DSL、JavaScript bridge 和 Wasm 等扩展入口，并包含包解析、生命周期管理、插件 SDK 与面向作者的 TypeScript 声明。这些入口为替换和扩展运行时提供了基础，但接口仍在演进，并非所有模块都已完成插件化。[1]

插件跨设备运行的前提，是明确声明所需运行时、Host 能力、权限、兼容版本、状态范围和恢复信息。我们希望兼容节点据此执行或接续任务，同时保留平台特有能力的边界。下表是下一阶段 SDK 契约的建议维度，尚非稳定规范；当前不能假定任意插件都能在所有设备上运行。

SDK 契约建议	建议描述内容	跨节点价值
身份与兼容性	包标识、版本、来源、Core 与 SDK 兼容范围。	让节点知道是否可以解析、加载和迁移结果。
运行时与依赖	JavaScript、Wasm 或其他运行时，所需服务和资源。	避免把平台假设伪装成通用能力。
能力与权限	Host capability、工具 effect、敏感操作和批准需求。	使调度与用户确认基于真实风险。
状态与接续	本地、任务或 Space 状态，产物和恢复信息。	让任务在兼容节点间留下可继续的轨迹。



研究语境与技术立场

大语言模型驱动的 Agent 通常把感知、推理、行动和记忆组合在一个循环中。Xi 等人的综述将 LLM Agent 概括为以模型为核心、与感知和行动相结合的系统；这提供了理解 Agent 的有用框架，但没有替代产品级运行时必须解决的身份、权限、持久化和设备协作问题。[6] Operit2 的位置在于把这些运行时问题作为一等工程对象。

行动轨迹与可解释的任务事实

ReAct 说明，交错的推理轨迹和外部动作可以帮助模型维持任务计划，并让过程比只有最终答案更容易理解。[3] Operit2 不把模型内部思维链当作应该无限同步的对象，却认可任务过程需要留下可验证的外部事实。因此，工具调用、输入、结果、错误、产物、执行节点和阶段总结在系统层面拥有重要地位。它们既服务于当前对话，也服务于跨设备查看、备份恢复和下一次继续。

模型上下文与外部记忆的分层

MemGPT 将有限上下文窗口与分层记忆管理联系起来，指出长对话与大文档需要在不同存储层之间选择信息。[4] Generative Agents 进一步展示了经验记录、反思和动态检索如何影响后续计划。[5] 这些研究启发我们把模型上下文看成稀缺的工作区，而不是用户整个数字生活的唯一载体。Operit2 不宣称解决模型内生的长期记忆问题；它要解决的是模型之外的连续性基础：工作状态、持久化个人与任务历史、设备世界状态、权限与数据所有权记录。

因此，跨设备保存记录与向模型提供上下文应当分开处理。主模型按当前任务需要接收摘要、证据和可访问引用，详细日志与原始产物在需要时读取。这一设计希望减少上下文噪声并保留追溯能力；摘要遗漏、检索质量和接续效果仍需通过实际任务验证。

模型进步之后仍然需要什么

更强的模型可能减少对专用提示词、固定 Agent 循环和手写工具规划的依赖。Operit2 需要为这些变化保留替换空间。同时，只要任务仍要在设备上执行，系统就需要记录可访问的资源、授权范围、任务结果以及设备更换后的恢复路径。未来 harness 的形式尚不确定，我们目前选择优先维护这些与用户直接相关的运行基础。

可靠性 迁移与预览期演进

Operit2 处于基础设施重构与稳定性打磨并行的阶段。我们不会用“永远兼容”阻止必要的架构改进，也不会把每一次重构看成可以无声丢失用户连续性的理由。接口、数据格式、插件契约和同步语义应当可版本化；当迁移发生时，用户应能够理解迁移内容、保留可恢复记录，并知道任务与设备关系是否仍然成立。



对个人用户而言，迁移应覆盖对话和任务历史、必要资料、扩展状态与可解释的任务轨迹。我们的目标是：当新手机、电脑或服务器加入时，只要仍有完整的同步副本或备份，就能尽量减少手工搬运。恢复仍取决于副本范围和格式兼容性，新节点需要重新确认身份、信任关系与本机权限；已经丢失且没有副本的运行细节无法凭空恢复。

可靠性需要用可观察的行为来衡量。长连接的重连与释放、取消传播、背压、流结束、任务去重、内容校验、失败转移、执行 generation 和恢复记录都属于需要长期验证的运行时品质。Rust 为 Core 提供了适合构建长运行系统的语言与生态基础，但语言本身不自动保证没有资源泄漏或生命周期问题；最终品质来自明确的生命周期、背压、取消、资源控制和可观测性。

实施重点与结语

接下来的重点是让用户在合适的设备上发起、查看、批准、继续和恢复 Agent 工作，减少反复配置与整理上下文的负担。当前优先修复问题、打磨稳定性，再逐步完善下列能力。

优先方向	要解决的真实问题	预期形成的基础能力
能力编排	跨设备暴露的能力还需要成为可理解、可选择、可执行的工具链。	节点可以基于契约、健康、历史证据与用户策略被选择。
长期运行品质	连续任务、流、连接、取消与资源释放需要经受长时间使用。	任务稳定性、可观测性和可恢复性得到系统化验证。
迁移与恢复	新设备加入或旧设备退出时，用户连续性不能依赖手工搬运。	同步、备份、恢复和选择性重建拥有清楚的用户路径。
插件 SDK	扩展作者需要知道如何声明能力、风险、状态和兼容性。	支持兼容节点执行或接续，并明确不兼容的条件。
人机授权体验	普通用户需要理解敏感同步和有副作用操作。	确认、撤销、轮换和策略选择成为自然交互，而非命令行负担。

我们希望每台设备保留自己擅长的能力，让用户换一台设备后，仍能接着做自己的事。手机便于随时交流，桌面适合创作与审核，长期在线的节点可以持续执行任务。Operit2 会继续经历必要的重构；我们愿意把精力放在清楚的权限、可靠的记录和可恢复的协作上，让普通人也能更容易地使用 Agent。



附录 源码基线与参考资料

源码基线

下表保留初版于 2026 年 9 月 7 日整理的源码与文档入口，供读者查阅工程依据。本文同时包含设计目标，目录位置与接口会随重构变化，不构成 API 稳定性承诺。

主题	源码或文档入口	用于校对的内容
CoreNode Space Binding	docs/core-node-space-binding-architecture.md core/crates/node/runtime/src/CoreNodeRouter.rs	节点平等、Space 范围、Binding 路由和单一执行者语义。
持久化同步	core/crates/node/runtime/src/ SpacePersistenceSyncService.rs	变更触发同步、直接配对节点、同步操作和文件传输。
Link 与 Access	docs/link-access-architecture.md	本地应用链路与 Space 节点链路的职责分离。
Host 能力	core/crates/foundation/host-api/src/HostManager.rs core/crates/foundation/host-api/src/lib.rs	HostManager、环境描述、平台能力和运行时 trait。
权限边界	docs/permission-access-architecture.md	运行环境、Host 授权、AI 能力模式与工具批准的分层。
插件运行时	core/CRATE_BOUNDARIES.md plugins/ core/crates/plugin/sdk/	ToolPkg、JavaScript、Wasm、SDK 和运行时边界。
历史起点	Operit 仓库的 terminal 模块 TerminalManager.kt LocalTerminalProvider.kt UbuntuDocumentsProvider.kt	Android 上的 Ubuntu PRoot、终端会话与 SAF 文件系统暴露。



参考资料

- [1] AAswordman. Operit2 source repository. GitHub. <https://github.com/AAswordman/Operit2> 访问日期 2026 年 9 月 7 日。
- [2] AAswordman. Operit source repository. GitHub. <https://github.com/AAswordman/Operit> 访问日期 2026 年 9 月 7 日。
- [3] Yao, S., Zhao, J., Yu, D., et al. ReAct Synergizing Reasoning and Acting in Language Models. ICLR 2023. <https://arxiv.org/abs/2210.03629>
- [4] Packer, C., Fang, V., Patil, S. G., et al. MemGPT Towards LLMs as Operating Systems. arXiv 2023. <https://arxiv.org/abs/2310.08560>
- [5] Park, J. S., O'Brien, J. C., Cai, C. J., et al. Generative Agents Interactive Simulacra of Human Behavior. UIST 2023. <https://arxiv.org/abs/2304.03442>
- [6] Xi, Z., Chen, W., Guo, X., et al. The Rise and Potential of Large Language Model Based Agents A Survey. arXiv 2023. <https://arxiv.org/abs/2309.07864>